

FULL STACK DEVELOPMENT PROJECT

Task – 1:

CodeAlpha_SimpleEcommerceStore

Tech Stack:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js + Express.js
- Database: MongoDB + Mongoose
- Authentication: JWT
- Password Hashing: bcrypt

Features:

- ☒ User Registration
- ☒ User Login
- ☒ Product Listing
- ☒ Product Details
- ☒ Shopping Cart
- ☒ Place Order
- ☒ MongoDB Database

Folder Structure:

CodeAlpha_SimpleEcommerceStore/

```
|
|
|└─ backend/
|
|  └─ server.js
|
|  └─ package.json
|
|  └─ models/
|
|    └─ User.js
|
|    └─ Product.js
```

```
| | Order.js
| |─ routes/
| | auth.js
| | products.js
| | orders.js
| └─ middleware/
|   auth.js
|
|─ frontend/
| index.html
| login.html
| register.html
| product.html
| cart.html
| css/style.css
| js/app.js
|
└─ README.md
```

Backend

Install

mkdir backend

cd backend

npm init -y

npm install express mongoose bcrypt jsonwebtoken cors dotenv

server.js

```
const express=require("express");

const mongoose=require("mongoose");

const cors=require("cors");


const app=express();


app.use(cors());
app.use(express.json());


mongoose.connect("mongodb://127.0.0.1:27017/codealpha");


app.use("/auth",require("./routes/auth"));
app.use("/products",require("./routes/products"));
app.use("/orders",require("./routes/orders"));


app.listen(5000,()=>{
  console.log("Server running");
});
```

models/User.js

```
const mongoose=require("mongoose");


module.exports=mongoose.model("User",{

  name:String,
  email:String,
  password:String
```

```
});
```

models/Product.js

```
const mongoose=require("mongoose");

module.exports=mongoose.model("Product",{

name:String,
price:Number,
description:String,
image:String

});
```

models/Order.js

```
const mongoose=require("mongoose");

module.exports=mongoose.model("Order",{

userId:String,
products:Array,
total:Number

});
```

routes/auth.js

```
const router=require("express").Router();
```

```
const User=require("../models/User");
```

```
const bcrypt=require("bcrypt");
```

```
const jwt=require("jsonwebtoken");
```

```
router.post("/register",async(req,res)=>{
```

```
const hash=await bcrypt.hash(req.body.password,10);
```

```
const user=new User({
```

```
name:req.body.name,
```

```
email:req.body.email,
```

```
password:hash
```

```
});
```

```
await user.save();
```

```
res.json({message:"Registered"});
```

```
});
```

```
router.post("/login",async(req,res)=>{
```

```
const user=await User.findOne({email:req.body.email});
```

```
if(!user) return res.sendStatus(404);
```

```
const ok=await bcrypt.compare(req.body.password,user.password);
```

```
if(!ok) return res.sendStatus(401);
```

```
const token=jwt.sign({id:user._id},"secret");
```

```
res.json({token});
```

```
});
```

```
module.exports=router;
```

routes/products.js

```
const router=require("express").Router();
```

```
const Product=require("../models/Product");
```

```
router.get("/",async(req,res)=>{
```

```
const products=await Product.find();
```

```
res.json(products);
```

```
});
```

```
router.post("/", async(req, res) => {  
  
  const product = new Product(req.body);  
  
  await product.save();  
  
  res.json(product);  
  
});  
  
module.exports = router;
```

routes/orders.js

```
const router = require("express").Router();  
  
const Order = require("../models/Order");  
  
router.post("/", async(req, res) => {  
  
  const order = new Order(req.body);  
  
  await order.save();  
  
  res.json({message: "Order Placed"});  
  
});  
  
module.exports = router;
```

Frontend

index.html

```
<!DOCTYPE html>

<html>

<head>

<link rel="stylesheet" href="css/style.css">

</head>

<body>


<h1>Simple E-Commerce Store</h1>


<div id="products"></div>


<script src="js/app.js"></script>


</body>

</html>
```

style.css

```
body{

font-family:Arial;

padding:20px;

background:#f5f5f5;

}


.card{

background:white;
```



```
padding:20px;
margin:20px;
border-radius:10px;
box-shadow:0 0 10px gray;
}
```

```
button{
padding:10px;
background:#2196f3;
color:white;
border:none;
cursor:pointer;
}
```

app.js

```
fetch("http://localhost:5000/products")

.then(res=>res.json())

.then(data=>{

const div=document.getElementById("products");

data.forEach(p=>{

div.innerHTML+=`

<div class="card">
```

```
<h2>${p.name}</h2>
```

```
<p>${p.description}</p>
```

```
<h3>₹${p.price}</h3>
```

```
<button>Add To Cart</button>
```

```
</div>
```

```
`;
```

```
});
```

```
});
```

MongoDB Sample Product

```
{  
  "name": "Laptop",  
  "price": 45000,  
  "description": "Core i5 16GB RAM",  
  "image": "laptop.jpg"  
}
```

README

Features

User Registration

User Login

JWT Authentication

MongoDB Database

Shopping Cart

Place Order

REST API

Responsive Frontend

Task - 2:

Social Media Platform

Features:

- User Registration & Login
- User Profiles
- Create Posts
- Comments
- Like System
- Follow/Unfollow Users
- Frontend: HTML, CSS, JavaScript
- Backend: Express.js (or Django)

- Database: MongoDB

Recommended Project Structure

CodeAlpha_SocialMediaPlatform/

```
|
|
|└─ backend/
|  |└─ server.js
|  |└─ package.json
|  |└─ config/
|  |  |db.js
|  |└─ models/
|  |  |User.js
|  |  |Post.js
|  |  |Comment.js
|  |└─ routes/
|  |  |auth.js
|  |  |users.js
|  |  |posts.js
|  |└─ middleware/
|  |  |auth.js
|  |└─ uploads/
|
|└─ frontend/
|  |└─ index.html
|  |└─ login.html
|  |└─ register.html
|  |└─ profile.html
|  |└─ css/
```

```
| | └─ js/
| | └─ images/
|
└─ README.md
```

Features

- ☒ JWT Authentication
- ☒ Register/Login
- ☒ User Profiles
- ☒ Edit Profile
- ☒ Create Posts
- ☒ Delete Posts
- ☒ Comment on Posts
- ☒ Like/Unlike Posts
- ☒ Follow/Unfollow Users
- ☒ News Feed
- ☒ MongoDB Database
- ☒ Responsive UI

Backend

- Express.js
- MongoDB (Mongoose)
- JWT Authentication
- bcrypt Password Hashing
- Multer (profile image upload)

Database Collections

users

posts

comments

followers

likes

REST APIs

POST /auth/register

POST /auth/login

GET /users/profile

PUT /users/profile

POST /posts

GET /posts

DELETE /posts/:id

POST /posts/:id/comment

POST /posts/:id/like

POST /users/follow/:id

POST /users/unfollow/:id

Home Page

Logo Search Logout

Create Post

[Text Box]

[Upload Image]

[Post]

John

Hello Everyone

 25 Likes

 8 Comments

[Like] [Comment]

User Profile

Profile Photo

Name

Bio

Followers

Following

Posts

Technologies

Frontend

HTML

CSS

JavaScript

Backend

Node.js

Express.js

Database

MongoDB

Authentication

JWT

bcrypt

Task – 3:

Project Management Tool

- User Authentication
- Create Group Projects
- Task Boards
- Task Cards
- Assign Tasks
- Comments
- Backend to manage users, projects, tasks, and comments
- **Bonus:** Real-time notifications using WebSockets

Project Name

CodeAlpha_ProjectManagementTool

Tech Stack

Frontend	Backend	Database	Authentication
React.js Node.js + Express MongoDB JWT			

Features

Authentication

- Register
- Login
- Logout
- JWT Authentication

Dashboard

- View all projects
- Create Project
- Delete Project
- Edit Project

Boards

- To Do
- In Progress
- Review
- Completed

Tasks

- Create Task
- Edit Task

- Delete Task
- Assign User
- Due Date
- Priority
- Status

Comments

- Add Comment
- Delete Comment

Users

- Invite Members
- Project Roles

Bonus

- Socket.io Notifications
- Drag & Drop (react-beautiful-dnd)
- Dark Mode
- Search Tasks

Folder Structure

CodeAlpha_ProjectManagementTool/

backend/

|

├─ server.js

├─ package.json

├─ .env

├─ config/

| db.js

├─ middleware/

```
|   auth.js
|   └── models/
|       User.js
|       Project.js
|       Task.js
|       Comment.js
|   └── routes/
|       auth.js
|       projects.js
|       tasks.js
|       comments.js
└── sockets/
    socket.js
```

frontend/

```
|
|   └── src/
|       ├── components/
|       |   Navbar.jsx
|       |   Sidebar.jsx
|       |   ProjectCard.jsx
|       |   TaskCard.jsx
|       |   CommentBox.jsx
|       ├── pages/
|       |   Login.jsx
|       |   Register.jsx
|       |   Dashboard.jsx
|       |   Board.jsx
```

```
| | Project.jsx
| |─ App.jsx
| └─ main.jsx
|
└─ package.json
```

MongoDB Collections

users

projects

tasks

comments

notifications

Database Schema

User

```
{
  name,
  email,
  password,
  avatar
}
```

Project

```
{
  title,
  description,
  owner,
  members,
  createdAt
```

```
}
```

Task

```
{
```

```
  title,
```

```
  description,
```

```
  assignedTo,
```

```
  priority,
```

```
  dueDate,
```

```
  status,
```

```
  projectId
```

```
}
```

Comment

```
{
```

```
  taskId,
```

```
  userId,
```

```
  text,
```

```
  createdAt
```

```
}
```

REST API

Authentication

POST /api/auth/register

POST /api/auth/login

Projects

GET /api/projects

POST /api/projects

PUT /api/projects/:id

DELETE /api/projects/:id

Tasks

GET /api/tasks/:projectId

POST /api/tasks

PUT /api/tasks/:id

DELETE /api/tasks/:id

Comments

POST /api/comments

GET /api/comments/:taskId

Dashboard UI

Logo

Projects

+ New Project

Website Project

Members : 5

Tasks : 18

Mobile App

Members : 3

Tasks : 22

Board UI

TO DO

Task 1

Task 2

IN PROGRESS

Task 3

Task 4

REVIEW

Task 5

COMPLETED

Task 6

Required NPM Packages

Backend

express

mongoose

jsonwebtoken

bcrypt

cors

dotenv

socket.io

Frontend

react

react-router-dom

axios

socket.io-client

react-beautiful-dnd

Extra Features

- Email notifications
- File uploads
- Calendar view
- Activity log
- User profile
- Team management
- Responsive UI
- Dashboard analytics
- Task filters
- Search
- Drag-and-drop Kanban board

README

Project Management Tool

Features

✓ Authentication

✓ Create Projects

✓ Kanban Board

✓ Task Assignment

✓ Due Dates

✓ Comments

✓ Team Collaboration

✓ JWT Authentication

✓ MongoDB

✓ Responsive Design

Bonus

✓ Socket.io Notifications

✓ Drag & Drop

Task – 4:

Real-Time Communication App

- Multi-user Video Calling
- Screen Sharing

- File Sharing
- Whiteboard
- User Authentication
- Data Encryption
- WebRTC + Socket.io/Firebase